Báo cáo huấn luyện T1.3 — YOLOv3-Tiny 4W4A trên VisDrone (car)

Oct 16, 2026

Tóm tắt

Lần huấn luyện T1.3 đầu tiên đạt **mAP@0.5 = 0.0173**, thấp hơn khoảng 10 lần so với 0.1768 của bài báo (Table 5). Mô hình có phát hiện được xe, nhưng định vị hộp kém: cùng checkpoint đạt mAP 0.238 ở IoU 0.1.

Nguyên nhân chính nằm ở hàm loss: loss MSE tự xây dựng không khớp với loss YOLOv5 mà decode của tác giả ngụ ý. Loss đã được thay bằng YOLOv5 ComputeLoss điều chỉnh cho HardTanh (commit `30aa235`), nhưng **chưa được huấn luyện lại**.

Cấu hình huấn luyện

Run `runs/pruned_4w4a`: mạng YOLOv3-Tiny đã cắt tỉa theo Table 1 của bài báo, lượng tử hoá 4W4A, huấn luyện QAT từ đầu.

Mục	Giá trị	Nguồn
Kiến trúc	10 lớp conv, 350.696 trọng số, 1 nhánh phát hiện 13×13 (stride 32), 3 anchor (10×14, 23×27, 37×58)	Paper Table 1; anchor từ repo tác giả
Lượng tử hoá	Conv đầu và cuối 8 bit, các lớp giữa 4 bit trọng số / 4 bit kích hoạt; đầu ra QuantHardTanh INT8, scale 1/127	Paper §3.1.1; repo
BatchNorm	Bật	Giả định A15 (mạng không BN không học được)
Dữ liệu	VisDrone2019-DET, chỉ lớp `car`; train 6.471 ảnh / 144.866 hộp, val 548 ảnh / 14.064 hộp	Paper §3.1.2
Ảnh đầu vào	416×416, letterbox	Paper Table 3; A4
Augmentation	HSV + lật ngang; không mosaic/scale/translate	A4
Tối ưu	SGD, lr0 0.01, momentum 0.937, weight decay 5e-4, OneCycleLR	Paper Table 3; hyp.scratch-low (A5)

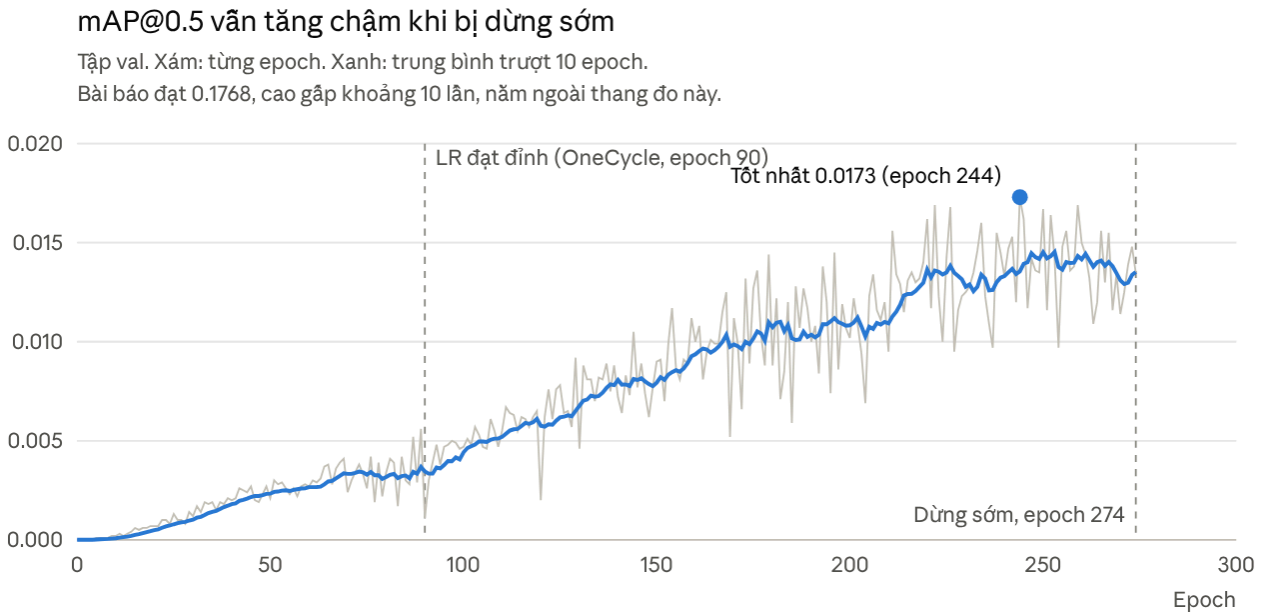
Mục	Giá trị	Nguồn
Epoch / batch / patience	300 / 64 / 30	Paper Table 3
Loss	MSE trên target nghịch đảo từ decode, hệ số box 0.05, obj 1.0	Tự xây dựng (đã thay sau run này)
Đánh giá	Tập val, conf 0.001, NMS IoU 0.6, tối đa 300 hộp; P/R tại F1 lớn nhất	D3; mặc định Ultralytics
Phần cứng	1 × TITAN RTX 24 GB, khoảng 33 s/epoch	

Kết quả

Run dừng sớm ở epoch 274/300 vì patience 30; checkpoint tốt nhất ở epoch 244. Mọi chỉ số đều thấp hơn bài báo nhiều lần.

Chỉ số	Run này (epoch 244)	Paper Table 5 (pruned)
Precision	0.1245	0.4816
Recall	0.0737	0.1716
mAP@0.5	0.0173	0.1768
Box loss	0.0895	0.1531
Obj loss	0.0825	0.3049

Giá trị loss của hai cột khác thang đo (MSE so với CIoU/BCE) nên không so trực tiếp được. Số liệu bài báo chỉ dùng làm bối cảnh, không phải mục tiêu tối ưu.



runs/pruned_4w4a/results.json · 275 epoch (0–274), tập val

Trung bình mAP theo từng khoảng 30 epoch vẫn tăng: 0.0100 (epoch 150–179), 0.0107, 0.0131, rồi 0.0139 (epoch 240–274). Val chỉ có 548 ảnh nên các epoch liền kề dao động từ 0.010 đến 0.017, làm patience kích hoạt khi xu hướng vẫn đi lên.

Chẩn đoán

Chấm lại `best.pt` (epoch 244) trên tập val ở nhiều ngưỡng IoU cho thấy lỗi nằm ở định vị, không phải ở việc tìm xe.

Ngưỡng IoU	Precision	Recall	mAP
0.5	0.119	0.076	0.017
0.3	0.275	0.191	0.087
0.1	0.460	0.327	0.238

- **Định vị:** với mỗi xe thật, hộp dự đoán khớp nhất có IoU trung vị 0.077; chỉ 9% số xe có hộp đạt IoU > 0.5.
- **Kích thước xe:** trung vị 11.9 × 8.6 px trên ảnh 416, 33% xe rộng dưới 8 px. Tâm hộp lệch 0.2 ô lưới (≈ 6 px) đã đủ làm IoU về gần 0.
- **Va chạm ô lưới không phải giới hạn chính:** trung bình 1.86 xe mỗi ô có xe, trong khi mỗi ô có 3 anchor.
- **Đầu ra:** kênh obj bão hoà ở ±1 tại 88% vị trí; các kênh xywh chỉ 3–4%. QuantHardTanh cho gradient bằng 0 khi đầu ra vượt ±1 (đã đo).

Nguyên nhân

Decode của repo tác giả chính là decode của YOLOv5 khi thay `sigmoid(x)` bằng `p = y/2 + 0.5`:

$2p - 0.5 = y + 0.5$ và **$(2p)^2 = (y + 1)^2$**

Kết hợp với câu "standard low-scratch hyper-parameters" ở §3.1.4, nhiều khả năng tác giả huấn luyện bằng YOLOv5 ComputeLoss với cùng phép thay thế. Loss của run này khác ở ba điểm:

Thành phần	Run này	YOLOv5
Box	MSE trên xywh nghịch đảo, hệ số 0.05	1 − CIoU trên hộp đã decode, hệ số $0.05 \cdot 3/nl = 0.15$
Objectness	MSE, target = 1	BCE, target = IoU của hộp dự đoán
Gán target	Chỉ ô chứa tâm	Ô chứa tâm + 2 ô lân cận gần nhất

Hệ số box 0.05 trong `hyp.scratch-low` được hiệu chỉnh cho CIoU, nên khi dùng với MSE thì tín hiệu định vị quá yếu. Giá trị loss cuối cũng khớp với giả thuyết này: bài báo báo cáo box 0.153 và obj 0.305, đúng thang đo CIoU/BCE, còn run này là 0.089 và 0.080 theo thang MSE.

Đã sửa

Commit `30aa235` trên `main` thay `utils/loss.py` bằng YOLOv5 ComputeLoss điều chỉnh cho decode HardTanh:

- **Box:** 1 − CIoU trên hộp đã decode, gán 3 ô (ô chứa tâm + 2 ô lân cận), lọc anchor theo tỉ lệ `anchor_t` = 4.
- **Objectness:** BCE trên `p(y_obj)`, target = CIoU (detach, kẹp ⩾ 0).
- **Hệ số:** box 0.15, obj 1.27 nhân hệ số cân bằng 4.0; tổng loss nhân batch size, như YOLOv5.
- **`train.py`:** cắt gradient norm ở 10 như YOLOv5 v7.0.
- **Kiểm thử:** 10 test mới trong `tests/test_loss.py`, trong đó có test kiểm tra hộp mà loss chấm điểm trùng với hộp `Detect` trả ra lúc inference. Một lần forward/backward trên batch thật cho loss hữu hạn.

Hai giả định mới, khác YOLOv5:

- **Ép `p` vào [0.001, 0.999] trước khi lấy log.** Với `p` affine, gradient BCE xấp xỉ 1/p và sẽ nổ khi p ≈ 0. Không dùng clamp vì sẽ thêm một vùng gradient bằng 0.
- **Prior objectness** theo `_initialize_biases` của YOLOv5 ($p = 8/13^2$), đặt vào BN bias của conv cuối. Trên một batch thật lúc khởi tạo, obj loss giảm từ 8.05 xuống 2.62.

Cột box/obj trong log của run mới sẽ theo thang của YOLOv5, nên so trực tiếp được với Table 5 của bài báo nhưng không so được với run này.

Hạn chế và bước tiếp theo

Loss mới chưa được kiểm chứng bằng huấn luyện; chưa có số liệu nào cho thấy mAP sẽ cải thiện.

- [ ] Huấn luyện lại với `--name` mới (ví dụ `pruned_4w4a_ciou`), không dùng `--resume`; giữ `runs/pruned_4w4a` làm bằng chứng.
- [ ] Nếu mAP vẫn thấp: thay `OneCycleLR` (warmup 30%, tự đổi momentum) bằng lịch của YOLOv5 (warmup 3 epoch rồi giảm tuyến tính); các tham số `warmup_*` trong file hyp hiện chưa được dùng.
- [ ] Xem xét bật mosaic/scale/translate; hiện đang tắt có chủ đích theo A4 để hình học lúc train và inference giống nhau.
- [ ] Val chỉ có 548 ảnh nên mAP dao động mạnh giữa các epoch; patience 30 có thể dừng sớm khi đường cong vẫn đang tăng.
- [ ] Tính thêm số liệu trên tập train (D3), vì bài báo tự mâu thuẫn về việc Table 5 là val hay train.